

# Tasks › Run-Application ›

## Run-Single-Instance-Stateful-Application

Run a Single-Instance Stateful Application | Kubernetes

Run a Single-Instance Stateful Application This page shows you how to run a single-instance stateful application in Kubernetes using a PersistentVolume and a Deployment. The application is MySQL. Objectives Create a PersistentVolume referencing a disk in your environment. Create a MySQL Deployment. Expose MySQL to other pods in the cluster at a known DNS name. Before you begin You need to have a Kubernetes cluster, and the kubectl command-line tool must be configured to communicate with your cluster. It is recommended to run this tutorial on a cluster with at least two nodes that are not acting as control plane hosts. If you do not already have a cluster, you can create one by using minikube

or you can use one of these Kubernetes playgrounds: iximiuz Labs Killercoda KodeKloud To check the version, enter kubectl version. You need to either have a dynamic PersistentVolume provisioner with a default StorageClass,

or statically provision PersistentVolumes

yourself to satisfy the PersistentVolumeClaims

used here. Deploy MySQL You can run a stateful application by creating a Kubernetes Deployment and connecting it to an existing PersistentVolume using a

PersistentVolumeClaim. For example, this YAML file describes a

Deployment that runs MySQL and references the PersistentVolumeClaim. The file

defines a volume mount for /var/lib/mysql, and then creates a

PersistentVolumeClaim that looks for a 20G volume. This claim is

satisfied by any existing volume that meets the requirements,

or by a dynamic provisioner. Note: The password is defined in the config yaml, and this is insecure. See Kubernetes Secrets

for a secure solution. application/mysql/mysql-deployment.yaml

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
name: mysql
```

```
spec:
```

```
ports:
```

```
- port: 3306
```

```
selector:
```

```
app: mysql
```

```
clusterIP: None
```

```
---
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
name: mysql
```

```
spec:
```

```
selector:
```

```
matchLabels:
```

```
app: mysql
```

```
strategy:
```

```
type: Recreate
```

```
template:
```

```
metadata:
```

```
labels:
```

```

app: mysql
spec:
containers:
- image: mysql:9
name: mysql
env:
# Use secret in real usage
- name: MYSQL_ROOT_PASSWORD
value: password
ports:
- containerPort: 3306
name: mysql
volumeMounts:
- name: mysql-persistent-storage
mountPath: /var/lib/mysql
volumes:
- name: mysql-persistent-storage
persistentVolumeClaim:
claimName: mysql-pv-claim
application/mysql/mysql-pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
name: mysql-pv-volume
labels:
type: local
spec:
storageClassName: manual
capacity:
storage: 20Gi
accessModes:
- ReadWriteOnce
hostPath:
path: "/mnt/data"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: mysql-pv-claim
spec:
storageClassName: manual
accessModes:
- ReadWriteOnce
resources:
requests:
storage: 20Gi
Deploy the PV and PVC of the YAML file:kubectl apply -f
https://k8s.io/examples/application/mysql/mysql-pv.yaml
Deploy the contents of the YAML file:kubectl apply -f
https://k8s.io/examples/application/mysql/mysql-deployment.yaml
Display information about the Deployment:kubectl describe deployment mysql
The output is similar to this:Name: mysql
Namespace: default
CreationTimestamp: Tue, 01 Nov 2016 11:18:45 -0700
Labels: app=mysql

```

```

Annotations: deployment.kubernetes.io/revision=1
Selector: app=mysql
Replicas: 1 desired | 1 updated | 1 total | 0 available | 1 unavailable
StrategyType: Recreate
MinReadySeconds: 0
Pod Template:
Labels: app=mysql
Containers:
mysql:
Image: mysql:9
Port: 3306/TCP
Environment:
MYSQL_ROOT_PASSWORD: password
Mounts:
/var/lib/mysql from mysql-persistent-storage (rw)
Volumes:
mysql-persistent-storage:
Type: PersistentVolumeClaim (a reference to a PersistentVolumeClaim in the same namespace)
ClaimName: mysql-pv-claim
ReadOnly: false
Conditions:
Type Status Reason
-----
Available False MinimumReplicasUnavailable
Progressing True ReplicaSetUpdated
OldReplicaSets: <none>
NewReplicaSet: mysql-63082529 (1/1 replicas created)
Events:
FirstSeen LastSeen Count From SubobjectPath Type Reason Message
-----
33s 33s 1 {deployment-controller } Normal ScalingReplicaSet Scaled up replica set mysql-63082529 to 1
List the pods created by the Deployment:kubectl get pods -l app=mysql
The output is similar to this:NAME READY STATUS RESTARTS AGE
mysql-63082529-2z3ki 1/1 Running 0 3m
Inspect the PersistentVolumeClaim:kubectl describe pvc mysql-pv-claim
The output is similar to this:Name: mysql-pv-claim
Namespace: default
StorageClass:
Status: Bound
Volume: mysql-pv-volume
Labels: <none>
Annotations: pv.kubernetes.io/bind-completed=yes
pv.kubernetes.io/bound-by-controller=yes
Capacity: 20Gi
Access Modes: RWO
Events: <none>
Accessing the MySQL instanceThe preceding YAML file creates a service that
allows other Pods in the cluster to access the database. The Service option
clusterIP: None lets the Service DNS name resolve directly to the
Pod's IP address. This is optimal when you have only one Pod
behind a Service and you don't intend to increase the number of Pods.Run a MySQL client to connect to the
server:kubectl run -it --rm --image=mysql:9 --restart=Never mysql-client -- mysql -h mysql -ppassword
This command creates a new Pod in the cluster running a MySQL client
and connects it to the server through the Service. If it connects, you
know your stateful MySQL database is up and running.Waiting for pod default/mysql-client-274442439-zyp6i to

```

be running, status is Pending, pod ready: false  
If you don't see a command prompt, try pressing enter.

mysql>

UpdatingThe image or any other part of the Deployment can be updated as usual with the `kubectl apply` command. Here are some precautions that are specific to stateful apps: Don't scale the app. This setup is for single-instance apps only. The underlying `PersistentVolume` can only be mounted to one Pod. For clustered stateful apps, see the `StatefulSet` documentation. Use `strategy: type: Recreate` in the Deployment configuration YAML file. This instructs Kubernetes to not use rolling updates. Rolling updates will not work, as you cannot have more than one Pod running at a time. The `Recreate` strategy will stop the first pod before creating a new one with the updated configuration. Deleting a deploymentDelete the deployed objects by name: `kubectl delete deployment,svc mysql`  
`kubectl delete pvc mysql-pv-claim`  
`kubectl delete pv mysql-pv-volume`

If you manually provisioned a `PersistentVolume`, you also need to manually delete it, as well as release the underlying resource.

If you used a dynamic provisioner, it automatically deletes the `PersistentVolume` when it sees that you deleted the `PersistentVolumeClaim`. Some dynamic provisioners (such as those for EBS and PD) also release the

underlying resource upon deleting the `PersistentVolume`. What's nextLearn more about Deployment objects. Learn more about Deploying applications `kubectl run` documentation Volumes and Persistent Volumes FeedbackWas this page helpful? Yes

NoThanks for the feedback. If you have a specific, answerable question about how to use Kubernetes, ask it on Stack Overflow.

Open an issue in the GitHub Repository if you want to report a problem

or

suggest an improvement. Last modified May 07, 2025 at 11:35 AM PST: Upgrade mysql image version used by Single-Instance Stateful tutorial fix yml (1ea9ecf3ad)